

PIDAC

Practicumhandleiding

Naam:



Minor Programmeren

Datarepresentaties, Tools en Technieken

Universiteit van Amsterdam

Versie: 3 (08-2025)

Auteur: Simon Pauw

Geïnspireerd door de syllabus *Digitale Techniek* van Ben Bruidegom

Met input van: Edwin Steffens en Puck te Rietmolen

Practicumregels



Wees voorzichtig met het gebruik van de PIDAC-modules. Deze modules worden niet meer geproduceerd en de losse onderdelen zijn vaak niet meer verkrijgbaar. Reparatie is meestal niet mogelijk. Houd je daarom aan de volgende regels:

Laat modules niet rondslingeren.

Pak een module direct uit de doos en plaats hem meteen op het experimenteerbord.

Let erop dat je de pinnen aan de achterkant niet buigt of breekt bij het plaatsen.

Leg modules die je niet meer nodig hebt terug in de doos waar ze vandaan kwamen.

Trek stroomdraadjes eruit bij de stekker en niet aan de draad zelf.

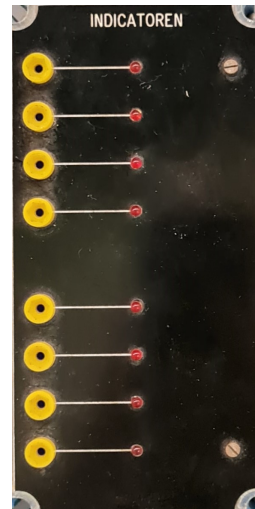
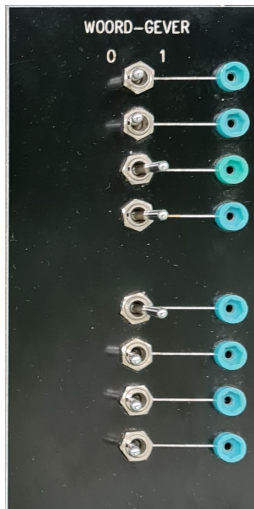
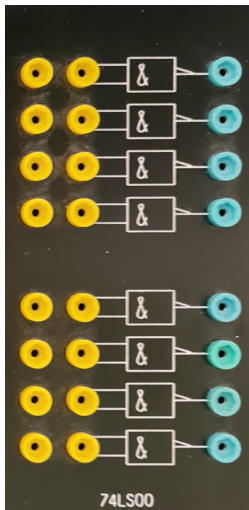
Week 1

Deel 1. Schakelingen en notaties

Je hebt maar een beperkt aantal logische poorten nodig om elke mogelijke schakeling te kunnen bouwen. Het doel van dit deel is om vertrouwd te raken met deze poorten en de manieren waarop we hiermee schakelingen kunnen bouwen. Je leert ook hoe je schakelingen symbolisch kunt beschrijven.

Logische poorten

De NAND-poort heeft het volgende gedrag: op de uitgang staat 5 volt, tenzij beide ingangen 5 volt ontvangen. Dan staat er 0 volt op de uitgang. Je kunt dit zelf testen met een woordgever en een indicator.



PIDAC Modules (vlnr): NAND, Woordgever, Indicator

Opdracht 1.1 Lees de practicumregels goed door voordat je begint.

Bouw een schakeling om te testen of de beschrijving van de NAND-poort klopt. (Nodig: NAND-module, woordgevermodule, indicatormodule en drie draadjes.)

Eentjes en nulletjes

Computers werken met eentjes en nulletjes. Elke poort kan in twee toestanden zijn: 5 volt (hoog, ofwel 1) of 0 volt (laag, ofwel 0).

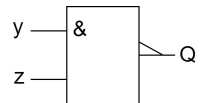
Niet alle digitale schakelingen werken op 5 volt; sommige gebruiken bijvoorbeeld 3,3 volt als hoog. Maar ongeacht het exacte voltage zijn er altijd twee toestanden: hoog en laag.

Notatie

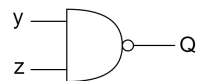
Het gedrag van een logische poort wordt beschreven door een waarheidstabel. Hiernaast zie je de waarheidstabel voor een NAND-poort met twee ingangen y en z en uitgang Q .

y	z	Q
0	0	1
0	1	1
1	0	1
1	1	0

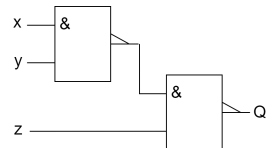
De symbolische notatie van een NAND-poort, zoals wij deze verder in dit document zullen gebruiken:



Er zijn verschillende notatiesystemen voor logische poorten, je zal voor de NAND-poort ook vaak dit alternatief tegenkomen:

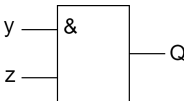
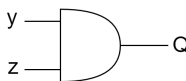
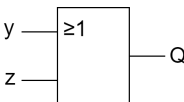
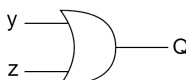
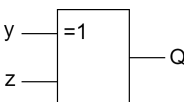
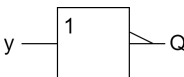
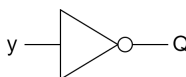
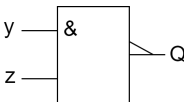
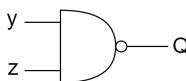
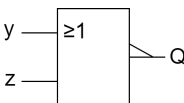
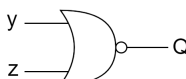


Complexere schakelingen kunnen worden opgebouwd door poorten met elkaar te verbinden:



Opdracht 1.2 Nodig: twee NAND-poorten, een woordgever, een indicator en vijf draadjes. Bouw de bovenstaande (complexe) schakeling en vul de waarheidstabel op het antwoordblad in.

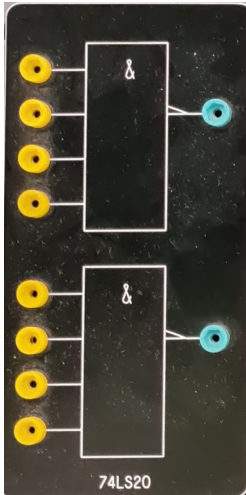
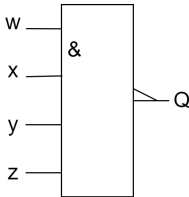
Veelvoorkomende poorten

Poort	Rechthoekige notatie	Alternatieve notatie	Waarheidstabel															
AND			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	y	z	Q	0	0	0	0	1	0	1	0	0	1	1	1
y	z	Q																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	y	z	Q	0	0	0	0	1	1	1	0	1	1	1	1
y	z	Q																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
XOR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	0	0	1	1	1	0	1	1	1	0
y	z	Q																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Inverter			<table><tr><th>y</th><th>Q</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	y	Q	0	1	1	0									
y	Q																	
0	1																	
1	0																	
NAND			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	1	0	1	1	1	0	1	1	1	0
y	z	Q																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	1	0	1	0	1	0	0	1	1	0
y	z	Q																
0	0	1																
0	1	0																
1	0	0																
1	1	0																

Overzicht van veelvoorkomende logische poorten

Meerdere ingangen

De bovenstaande poorten hebben allemaal twee ingangen (behalve de inverter), maar in principe kunnen dat er ook meer zijn. Het PIDAC-systeem bevat bijvoorbeeld ook NAND-poorten met vier ingangen. De schematische weergave en waarheidstabel van een NAND-poort met vier ingangen ziet er zo uit:

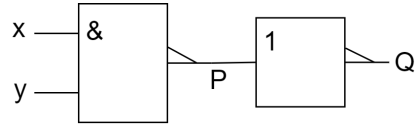
Module	Symbol	Waarheidstabel																																																																																					
		<table><tr><th><i>w</i></th><th><i>x</i></th><th><i>y</i></th><th><i>z</i></th><th><i>Q</i></th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>	<i>Q</i>	0	0	0	0	1	0	0	0	1	1	0	0	1	0	1	0	0	1	1	1	0	1	0	0	1	0	1	0	1	1	0	1	1	0	1	0	1	1	1	1	1	0	0	0	1	1	0	0	1	1	1	0	1	0	1	1	0	1	1	1	1	1	0	0	1	1	1	0	1	1	1	1	1	0	1	1	1	1	1	0
<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>	<i>Q</i>																																																																																			
0	0	0	0	1																																																																																			
0	0	0	1	1																																																																																			
0	0	1	0	1																																																																																			
0	0	1	1	1																																																																																			
0	1	0	0	1																																																																																			
0	1	0	1	1																																																																																			
0	1	1	0	1																																																																																			
0	1	1	1	1																																																																																			
1	0	0	0	1																																																																																			
1	0	0	1	1																																																																																			
1	0	1	0	1																																																																																			
1	0	1	1	1																																																																																			
1	1	0	0	1																																																																																			
1	1	0	1	1																																																																																			
1	1	1	0	1																																																																																			
1	1	1	1	0																																																																																			

Opdracht 1.3 Nodig: een module met NAND-poorten met vier ingangen, een woordgevermodule, een indicatormodule en vijf draadjes.

Experimenteer met de NAND-poort. Ga na of de bovenstaande tabel klopt. Wat gebeurt er als een ingang nergens op is aangesloten?

Poorten samenstellen

In principe is de bovenstaande verzameling poorten redundant. Dat wil zeggen, je kunt alle poorten maken met een combinatie van andere poorten.



Bijvoorbeeld, het schema rechtsboven heeft hetzelfde gedrag als een AND-poort.

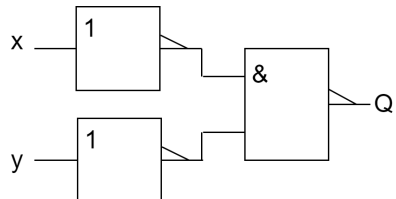
We kunnen dit zien aan de hand van de bijbehorende waarheidstabel.

x	y	P	Q
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Opdracht 1.4 Je kunt een NAND-poort als inverter gebruiken. Maak een schakeling die dat doet. Gebruik een woordgever en een indicator om je schakeling te testen. Teken het schema op het antwoordformulier en maak een waarheidstabel om te verifiëren dat het klopt.

Opdracht 1.5 Bouw het schema hiernaast na. Maak een waarheidstabel op je antwoordformulier. Met welke logische poort komt deze schakeling overeen?

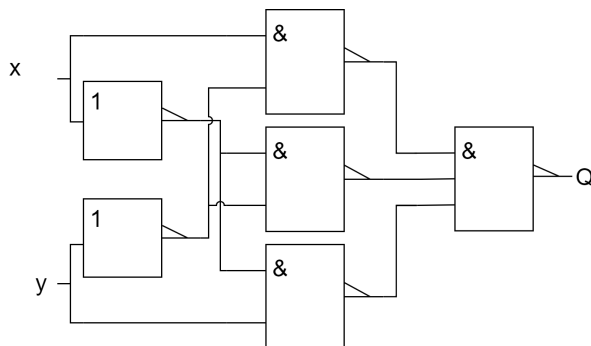
Tip: Er zijn geen inverters in PIDAC, maar daar kan je dus NAND-poorten voor gebruiken.



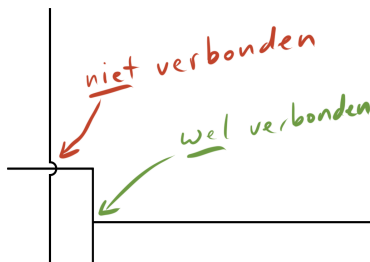
Opdracht 1.6 Maak een schakeling die het gedrag van een NAND-poort met drie ingangen nabootst. Gebruik twee NAND-poorten met twee ingangen en een inverter. Gebruik een woordgever en een indicator om je schakeling te testen. Teken het schema op het antwoordformulier en maak een waarheidstabel om te verifiëren dat het klopt.

Tip: je kunt een inverter (of NAND) en een NAND-poort gebruiken om een AND-poort te maken. Vervolgens kun je die AND-poort en een NAND-poort combineren tot een NAND-poort met drie ingangen.

Opdracht 1.7 Bouw het onderstaande schema na:



De verbindingen van de onderste inverter naar de bovenste twee NAND-poorten bevatten boogjes. Deze geven aan dat de betreffende verbinding een andere kruist zonder ermee verbonden te zijn:



Maak een waarheidstabel op je antwoordformulier. Vereenvoudig daarna de schakeling. Dat wil zeggen: maak een schakeling die minder poorten gebruikt, maar hetzelfde doet. Teken het schema op je antwoordformulier.

Voor deze schakeling heb je een NAND-poort met drie ingangen nodig. Er is geen PIDAC-module die zo'n poort heeft, maar je kunt hiervoor een NAND-poort met vier ingangen gebruiken.

Aftekenen! Laat opdrachten 1.1 - 1.7 aftekenen.

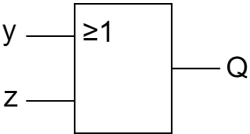
Boole-notatie

Het bouwen van simpele schakelingen kan vaak op basis van intuïtie en uitproberen. Maar bij complexere schakelingen werkt dat niet meer. Daarvoor gebruiken we Boole-algebra. Boole-algebra is in 1854 ontwikkeld door George Boole.

Als je ervaring hebt met propositielogica, zal Boole-algebra je bekend voorkomen. Het is namelijk equivalent — alleen de notatie verschilt.

We duiken deze week nog niet volledig in de algebra, maar we leren hoe je Boole-notatie kunt gebruiken om schakelschema's te beschrijven.

Voor een OR-poort gebruiken we bijvoorbeeld het symbool $+$. Dus het schema hiernaast schrijven we als:
 $Q = y + z$



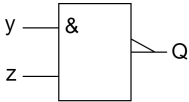
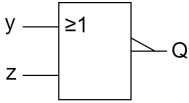
Hieronder een overzicht van de notatie van de verschillende poorten:

Poort	Boole-notatie	Schema
OR	$Q = y + z$	
AND	$Q = y \cdot z$	
Inverter	$Q = \overline{y}$	
XOR	$Q = y \oplus z$	

Let op: je hebt het symbool $\oplus$ strikt genomen niet nodig; je kunt XOR ook beschrijven met behulp van $\cdot$, $+$ en $\overline{}$.

Gebruik Boole-notatie

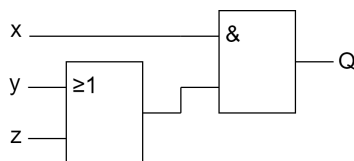
We kunnen ook NAND- en NOR-poorten beschrijven met een combinatie van operatoren:

Poort	Boole-notatie	Schema
NAND	$Q = \overline{y \cdot z}$	
NOR	$Q = \overline{y + z}$	

Complexe schakelingen

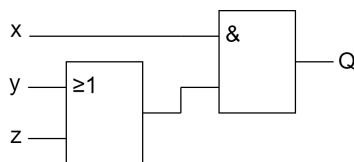
Complexe schakelingen kunnen we beschrijven met een combinatie van operatoren. Bijvoorbeeld, de schakeling hiernaast schrijven we als:

$$Q = x \cdot (y + z)$$



Een andere voorbeeld, voor de schakeling hiernaast is de expressie:

$$Q = x + \overline{y \cdot z}$$



Operatorvolgorde

Bij Boole-algebra zijn er net als in de wiskunde voorrangregels regels voor operatoren. Als er geen haakjes staan, gelden de volgende voorrangregels:

1. **Negatie** ($\bar{a}$) heeft altijd voorrang.
2. **AND** ($a \cdot b$) komt na negatie maar voor OR.
3. **OR** ($a + b$) en eventueel XOR ($a \oplus b$) komen als laatste.
4. Bij gelijke operatoren wordt van links naar rechts geëvalueerd.

Bijvoorbeeld, bij $a + b \cdot c$, evalueren we eerst $b \cdot c$ en daarna pas $a +$.

Opdracht 1.8 Bekijk de expressie: $Q = \overline{\bar{y} \cdot z}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Opdracht 1.9 Bekijk de expressie: $Q = \overline{(a \oplus b) \cdot c}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Opdracht 1.10 Bekijk de expressie: $Q = \overline{\overline{p} \cdot \overline{q} \cdot p \cdot q}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Aftekenen! Laat opdrachten 1.8 - 1.10 aftekenen.

Boole-expressies evalueren

Je kunt aan de hand van Boole-expressies direct een waarheidstabel opstellen, zonder de schakeling eerst te bouwen. Voor eenvoudige expressies zoals $Q = y + z$, $Q = y \cdot z$ en $Q = \bar{y}$ kun je de waarheidstabellen van de poorten raadplegen (zie de cheatsheet).

Voor complexere expressies vul je gewoon 1 of 0 in voor de variabelen en kijk je wat er uitkomt. Neem bijvoorbeeld $Q = \overline{y + z}$. Dit is de expressie van een NOR-poort.

Wat gebeurt er als $y = 1$ en $z = 0$?

$$Q = \overline{y + z} = \overline{1 + 0} = \overline{1} = 0$$

Als je dit alle combinaties van y en z doet, krijg je de volgende tabel:

y	z	Q
0	0	1
0	1	0
1	0	0
1	1	0

Opdracht 1.11 We hebben een schakeling gegeven door: $Q = \overline{v \cdot \overline{w}} \oplus p$

Beredeneer de waarheidstabel. Als je klaar bent, teken het schema, bouw het en controleer of je waarheidstabel klopt.

Opdracht 1.12 We hebben een schakeling gegeven door: $Q = \overline{\overline{a} + \overline{b}} + c$

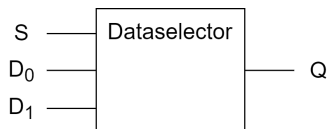
Beredeneer de waarheidstabel voor deze schakeling.

Opdracht 1.13 NAND-poorten hebben de handige eigenschap dat je er elke andere poort mee kunt bouwen. Ontwerp een schakeling van alleen NAND-poorten die zich gedraagt als een AND-poort. Teken het schema in je antwoordformulier.

Aftekenen! Laat opdrachten 1.11 - 1.13 aftekenen.

Opdracht 1.14

Een veelgebruikte schakeling is de dataselector (multiplexer). Stel je een systeem voor met twee datasignalen en een digitale selectielijn. Afhankelijk van de waarde van de selectielijn gaat het ene of het andere signaal door.



Inputs: D_0 , D_1 , S (select). Output: Q

S	D_0	D_1	Q
0	d_0	d_1	d_0
1	d_0	d_1	d_1

Volledig uitgewerkt:

S	D_0	D_1	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Ontwerp **en bouw** een schakeling voor deze tabel.

Dit kan een flinke puzzel zijn, vraag gerust om hulp van de docent of medestudenten.

Aftekenen! Laat opdracht 1.14 aftekenen.

Opmerkingen

Relatie met programmeertalen

De logica is in programmeertalen is direct gebaseerd op Boole-algebra. Je kan de besproken operatoren dus ook direct terugvinden in verschillende programmeertalen. De notatie is alleen net anders. Bijvoorbeeld:

- In plaats van 1 en 0 zie je vaak `true` en `false`
- $a + b$ wordt `a || b` in C, of `a or b` in Python
- $a \cdot b$ wordt `a && b` in C, of `a and b` in Python
- $\bar{a}$ wordt `!a` in C, of `not a` in Python

Relatie met propositielogica

Propositielogica is hetzelfde als Boole-algebra, alleen worden er conventioneel andere symbolen gebruikt:

- 1 en 0 komen overeen met $\top$ (waar) en $\perp$ (onwaar)
- $a + b$ wordt $a \vee b$
- $a \cdot b$ wordt $a \wedge b$
- $\bar{a}$ wordt $\neg a$

Week 2

Boolealgebra: de kracht van abstractie

In deze sessie leer je hoe je Boole-algebra kunt gebruiken om schakelingen te ontwerpen. In de vorige sessie heb je geleerd wat het verband is tussen een Boole-expressie en een schakeling. De kracht van Boole-algebra is dat je ermee kunt rekenen: je kunt het gebruiken om systematisch schakelingen te ontwerpen en te vereenvoudigen.

In het eerste deel van deze sessie introduceren we de rekenregels van de Boole-algebra. Daarna gebruiken we deze regels om schakelingen te ontwerpen.

Opdracht 2.1 We hebben een schakeling gegeven door $Q = (a + \bar{a}) \cdot b$. Beredeneer de waarheidstabel voor deze schakeling. Teken het schema op je antwoordblad, bouw de schakeling en test of je waarheidstabel klopt.

De bovenstaande schakeling is onnodig ingewikkeld. We kunnen het eenvoudiger maken. Als je naar de waarheidstabel kijkt, zie je wellicht dat deze gelijk is aan $Q = b$. Dit kunnen we ook beredeneren:

Beschouw de waarheidstabel voor het deel $a + \bar{a}$, hiernaast. Met andere woorden: $a + \bar{a}$ is altijd 1. Dus kunnen we de expressie $Q = (a + \bar{a}) \cdot b$ vereenvoudigen naar $Q = 1 \cdot b$.

a	$a + \bar{a}$
0	1
1	1

De uitdrukking $Q = 1 \cdot b$ kunnen we verder vereenvoudigen, met de waarheidstabel hiernaast. Zoals je ziet is $1 \cdot b$ altijd gelijk aan b . Met andere woorden, $Q = 1 \cdot b = b$

b	$1 \cdot b$
0	0
1	1

Oftewel, we kunnen aan de hand van de waarheidstabellen laten zien dat $Q = (a + \bar{a}) \cdot b$ vereenvoudigd kan worden tot $Q = b$

Regels

We kunnen veel wetmatigheden vaststellen door schakelingen te bouwen en daar waarheidstabellen mee te maken. Hiernaast vind je een aantal van deze wetmatigheden:

1. $z + z = z$

2. $z \cdot z = z$

3. $\overline{\overline{z}} = z$

4. $z + \overline{z} = 1$

5. $z \cdot \overline{z} = 0$

.
6. $\overline{\overline{0}} = 1$

7. $\overline{\overline{1}} = 0$

8. $z + 0 = z$

9. $z + 1 = 1$

10. $z \cdot 0 = 0$

11. $z \cdot 1 = z$

Opdracht 2.2

Verifieer aan de hand van een waarheidstabel dat de rekenregel $z \cdot \bar{z} = 0$ klopt.

Je kan de bovenstaande regels gebruiken om, bijvoorbeeld de expressie $Q = p + \overline{p \cdot \bar{0}}$ vereenvoudigen tot $Q = p$:

$$\begin{aligned} Q &= p + \overline{p \cdot \bar{0}} \\ &= p + \overline{\bar{p} \cdot 1} \quad (\text{regel 6}) \\ &= p + \bar{\bar{p}} \quad (\text{regel 11}) \\ &= p + p \quad (\text{regel 3}) \\ &= p \quad (\text{regel 1}) \end{aligned}$$

Opdracht 2.3

Vereenvoudig de expressie $Q = \overline{a \cdot \bar{a}}$. Gebruik de bovenstaande wetten en schrijf alle stappen van je afleiding op. Vergelijk beide expressies met behulp van een waarheidstabel.

Opdracht 2.4

Vereenvoudig de expressie $Q = a \cdot \overline{a \cdot \bar{a}}$. Geef een volledige afleiding met verwijzing naar de gebruikte regels.

Er zijn een aantal wetmatigheden die overeenkomen met wat je in de wiskunde gewend bent.

Associativiteit

Voor een herhaling van dezelfde operator maakt de volgorde waarin je ze toepast niet uit:

$$\begin{aligned} 12. \quad (x + y) + z &= x + (y + z) \\ 13. \quad (x \cdot y) \cdot z &= x \cdot (y \cdot z) \end{aligned}$$

Commutativiteit

Je kunt de volgorde van de argumenten van de operatoren omdraaien:

$$\begin{aligned} 14. \quad y + z &= z + y \\ 15. \quad y \cdot z &= z \cdot y \end{aligned}$$

Distributieve wetten

Voor $\cdot$ en $+$ zijn de volgende distributieregels van toepassing:

$$\begin{aligned} 16. \quad x \cdot y + x \cdot z &= x \cdot (y + z) \\ 17. \quad (x + y) \cdot (x + z) &= x + y \cdot z \end{aligned}$$

Opdracht 2.5

Laat met waarheidstabellen zien dat distributieve wet 17 correct is.

Opdracht 2.6

De expressie $Q = c + (a \cdot b + 0) \cdot \bar{a}$ is equivalent aan $Q = c$. Geef de afleiding die laat zien dat dit correct is. Schrijf alle stappen van je afleiding op en geef aan welke regels je gebruikt.

Tip: Je hebt alleen de regels 5, 8, 10, 13 en 15 nodig.

Absorptie en De Morgan

De laatste wetmatigheden, geven de relaties tussen de verschillende operatoren:

$$\begin{aligned} 18. \quad y + \bar{y} \cdot z &= y + z \quad (\text{absorptie}) \\ 19. \quad \overline{y \cdot z} &= \bar{y} + \bar{z} \quad (\text{De Morgan}) \\ 20. \quad y \cdot \bar{z} + \bar{y} \cdot z &= y \oplus z \quad (\text{XOR}) \end{aligned}$$

Opdracht 2.7

Laat met waarheidstabellen zien dat regel 19 correct is.

Waarheidstabellen en expressies ontwerpen

Dat was een vrij theoretisch uitstapje, maar we gaan nu eindelijk zien hoe we deze inzichten kunnen gebruiken voor het ontwerpen van schakelingen. De truc die je gaat leren is hoe je Boole-algebra kunt gebruiken om op een systematische manier waarheidstabellen om te zetten in schakelingen.

Neem de volgende expressie met twee variabelen: $Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot \bar{z} + y \cdot z$

Deze uitdrukking is altijd waar. Ze bestaat uit vier losse disjuncten (delen gescheiden door $+$): $\bar{y} \cdot \bar{z}$, $\bar{y} \cdot z$, $y \cdot \bar{z}$ en $y \cdot z$. Voor elke combinatie van y en z is precies één disjunct gelijk aan 1 en de rest aan 0. Bijvoorbeeld: als $y = 0$ en $z = 1$, dan is $\bar{y} \cdot z = 1$, en de rest 0. Omdat het een OR-expressie is, is $Q = 1$.

Stel dat we willen dat Q altijd de waarde 1 heeft, behalve als $y = 1$ en $z = 0$. We schrappen dan gewoon de bijbehorende disjunct:

$$Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + \cancel{y \cdot \bar{z}} + y \cdot z = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot z$$

Opdracht 2.8 Maak een waarheidstabel en ga na dat de bovenstaande expressie altijd waar is, behalve als $y = 1$ en $z = 0$.

Aftekenen! Laat opdracht 2.1 - 2.8 aftekenen.

We kunnen dus voor elke invulling van y en z een expressie formuleren die alleen waar is voor die specifieke combinatie. Hiermee kunnen we expressies opstellen op basis van waarheidstabellen.

Stel dat we de waarheidstabel hiernaast willen implementeren. (We weten dat dit de waarheidstabel voor een NAND-poort is, maar laten we kijken of we dat ook af kunnen leiden.)

y	z	Q
0	0	1
0	1	1
1	0	1
1	1	0

Eerst schrijven voor elke regel **waar $Q = 1$** de expressie die alleen die regel waarmaakt. (Bijvoorbeeld, voor $y = 0$ en $z = 0$, is die expressie $\bar{y} \cdot \bar{z}$.)

y	z	Q	
0	0	1	$\bar{y} \cdot \bar{z}$
0	1	1	$\bar{y} \cdot z$
1	0	1	$y \cdot \bar{z}$
1	1	0	

En deze plaatsen we samen in één disjunctieve expressie:

$$Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot \bar{z}$$

We kunnen dit vereenvoudigen tot de expressie voor NAND:

$$\begin{aligned}
 Q &= \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot \bar{z} \\
 &= \bar{y} \cdot (\bar{z} + z) + y \cdot \bar{z} && \text{(regel 16)} \\
 &= \bar{y} \cdot (z + \bar{z}) + y \cdot \bar{z} && \text{(regel 14)} \\
 &= \bar{y} \cdot 1 + y \cdot \bar{z} && \text{(regel 4)} \\
 &= \bar{y} + y \cdot \bar{z} && \text{(regel 11)} \\
 &= \bar{y} + \bar{z} && \text{(regel 18)} \\
 &= \overline{y \cdot z} && \text{(regel 19)}
 \end{aligned}$$

Dataselector

Dit principe werkt voor elke waarheidstabel, ongeacht het aantal inputs. Neem bijvoorbeeld de dataselector van vorige week. Dit was een enorme puzzel, maar ook hiervoor kunnen we voor elke regel waar $Q = 1$ een expressie opstellen:

D_0	D_1	S	Q	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{D_0} \cdot D_1 \cdot S$ (expressie 1)
1	0	0	1	$D_0 \cdot \overline{D_1} \cdot \overline{S}$ (expressie 2)
1	0	1	0	
1	1	0	1	$D_0 \cdot D_1 \cdot \overline{S}$ (expressie 3)
1	1	1	1	$D_0 \cdot D_1 \cdot S$ (expressie 4)

Samen vormen ze:

$$Q = \overbrace{\overline{D_0} \cdot D_1 \cdot S}^{\text{expressie 1}} + \overbrace{D_0 \cdot \overline{D_1} \cdot \overline{S}}^{\text{expressie 2}} + \overbrace{D_0 \cdot D_1 \cdot \overline{S}}^{\text{expressie 3}} + \overbrace{D_0 \cdot D_1 \cdot S}^{\text{expressie 4}}$$

Vereenvoudiging:

$$\begin{aligned} Q &= \overbrace{\overline{D_0} \cdot D_1 \cdot S}^{\text{expressie 4}} + \overbrace{\overline{D_0} \cdot D_1 \cdot S}^{\text{expressie 1}} + \overbrace{D_0 \cdot D_1 \cdot \overline{S}}^{\text{expressie 3}} + \overbrace{D_0 \cdot \overline{D_1} \cdot \overline{S}}^{\text{expressie 2}} && \text{(regel 14)} \\ &= S \cdot D_1 \cdot D_0 + S \cdot D_1 \cdot \overline{D_0} + \overline{S} \cdot D_0 \cdot D_1 + \overline{S} \cdot D_0 \cdot \overline{D_1} && \text{(regel 15)} \\ &= S \cdot D_1 \cdot (D_0 + \overline{D_0}) + \overline{S} \cdot D_0 \cdot (D_1 + \overline{D_1}) && \text{(regel 16)} \\ &= S \cdot D_1 \cdot 1 + \overline{S} \cdot D_0 \cdot 1 && \text{(regel 4)} \\ &= S \cdot D_1 + \overline{S} \cdot D_0 && \text{(regel 11)} \end{aligned}$$

Opdracht 2.9 Vereenvoudig de expressie van de dataschakelaar verder zodat je een uitdrukking overhoudt die je met alleen NAND-poorten kunt maken.

Opdracht 2.10 Stel een uitdrukking op voor deze waarheidstabel en vereenvoudig deze naar een schakeling die je met 3 NAND-poorten kan bouwen.

p	q	r	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Meerdere outputs

Tot nu toe hebben we alleen schakelingen met één output gezien. Wat als we een schakeling willen maken met meerdere outputs? De makkelijkste manier is om voor elke output een aparte schakeling te ontwerpen.

Neem bijvoorbeeld een schakeling die twee 1-bits binaire getallen optelt: een *half adder*. De mogelijke berekeningen zijn:

<i>a</i>	<i>b</i>	<i>carry</i>	<i>sum</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Opdracht 2.11 Ontwerp een schakeling voor de *half adder*.

- Maak twee waarheidstabellen: één voor de linkerbit (*carry*), één voor de rechterbit (*sum*).
- Stel Boole-expressies op voor beide outputs.
- Vereenvoudig indien nodig de expressies. (Houd er rekening mee dat je alleen XOR-poorten en NAND-poorten te beschikking hebt.)
- Bouw de schakeling.

Aftekenen! Laat opdracht 2.9 - 2.11 aftekenen.

Week 3

Rekenen met logica

Half Adder

Ter herhaling: vorige week ben je geëindigd met een *half adder*, een schakeling waarmee je twee 1-bits getallen kunt optellen.

De vier mogelijke berekeningen met deze rekenmachine zijn:

$$0 + 0 = 00, \quad 0 + 1 = 01, \quad 1 + 0 = 01, \quad 1 + 1 = 10$$

Dit geeft de volgende gecombineerde waarheidstabel voor de linkerbit (*carry*) en rechterbit (*sum*):

i_1	i_2	<i>carry</i>	<i>sum</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Deze tabel kan je opsplitsen voor de *carry* en *sum* en vervolgens kan je hiermee de bijbehorende expressies vinden:

i_1	i_2	<i>carry</i>
0	0	0
0	1	0
1	0	0
1	1	1

$$carry = i_1 \cdot i_2$$

i_1	i_2	<i>sum</i>
0	0	0
0	1	1
1	0	1
1	1	0

$$\begin{aligned} sum &= \overline{i_1} \cdot i_2 + i_1 \cdot \overline{i_2} \\ &= i_1 \cdot \overline{i_2} + \overline{i_1} \cdot i_2 \quad (\text{regel 14}) \\ &= i_1 \oplus i_2 \quad (\text{regel 20}) \end{aligned}$$

Aangezien we geen AND-poort hebben zal je de *sum* met twee NAND-poorten moeten implementeren, maar ondanks dat is de *half adder* dus een vrij eenvoudige schakeling.

Full Adder

Helaas kunnen we een *half adder* niet gebruiken om een schakeling te bouwen om meer dan twee bits op te tellen. Hiervoor hebben we een *full adder* nodig.

Een *full adder* kan drie 1-bits getallen optellen. De output is een 2-bits getal.

Opdracht 3.1 Vul de waarheidstabel voor de *full adder* op het antwoordblad in.

Opdracht 3.2 Ga na met de waarheidstabel dat:

$$sum = i_1 \cdot \overline{i_2} \cdot \overline{i_3} + \overline{i_1} \cdot i_2 \cdot \overline{i_3} + \overline{i_1} \cdot \overline{i_2} \cdot i_3 + i_1 \cdot i_2 \cdot i_3$$

Opdracht 3.3 Bewijs met de waarheidstabel dat:

$$sum = (i_1 \oplus i_2) \oplus i_3$$

Opdracht 3.4 Geef aan de hand van de waarheidstabel de (niet vereenvoudigde) expressie voor de *carry*.

Opdracht 3.5 De expressie voor de *carry* kan worden herschreven als:

$$carry = \overline{\overline{i_1 \cdot i_2} \cdot \overline{(i_1 \oplus i_2)} \cdot i_3}$$

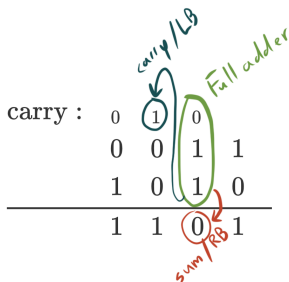
Bouw een *full adder* met twee XOR- en drie NAND-poorten. Label de poorten met i_1 , i_2 , i_3 , *carry* en *sum*.

Teken het schema op je antwoordblad.

Aftekenen! Laat opdracht 3.1 - 3.5 aftekenen.

Optelschakeling voor grotere getallen

Voor optelling van grotere getallen combineren we half en full adders.



Opdracht 3.6 Vind twee of drie andere groepjes en maak gezamenlijk een 4-bits optelschakeling met *full adders*.

Zorg voor labeling van inputs, outputs en volgorde van de adders.

Teken het schema op je antwoordblad. Je mag een *full adder* als een enkele module tekenen, met drie inputs en twee outputs. Als je dit doet, label de outputs van elke *full adder* (*sum* en *carry*).

Let op: er kan overflow ontstaan. Zorg voor een vijfde output die overflow aangeeft.

Aftekenen! Laat opdracht 3.6 aftekenen.

Optel-/aftrekschakeling

We bouwen nu een schakeling die zowel kan optellen als aftrekken met behulp van *two's complement* representatie.

Bij *two's complement* wordt aftrekken $A - B$ gerealiseerd door alle bits van B te flippen en vervolgens 1 op te tellen bij het eindresultaat. Oftewel, $A - B = A + \overline{B} + 1$, waarbij $\overline{B}$ wordt verkregen door alle bits van B te flippen.

De bijkomende moeilijkheid is dat we de bits van B alleen willen flippen als de $SUB/\overline{ADD}$ de waarde 1 heeft, want dan willen we aftrekken. (Als de $SUB/\overline{ADD}$ de waarde 0 heeft willen we optellen en moeten we de bits van B dus niet flippen.)

Dit geeft de volgende waarheidstabel per bit van B :

$SUB/\overline{ADD}$	Input bit	Output bit
0	0	0
0	1	1
1	0	1
1	1	0

Opdracht 3.7 Met welke logische poort kun je deze waarheidstabel realiseren?

Opdracht 3.8 Maak een 4-bits optel-/aftrekschakeling. De schakeling heeft 8 inputs en de $SUB/\overline{ADD}$ -selectiebit.

Teken het schema op je antwoordblad.

Let op: de overflow mag bij deze schakeling worden genegeerd.

Aftekenen! Laat opdracht 3.7 en 3.8 aftekenen.

